全体像：操作と処理を分けて読む

動作・ロジック解説 / 01　構成とデータの単位

01	02	03	04	05
React画面	認証付きAPI	処理・AI	保存	画面へ反映



対話の入口

返答を作る処理と、地図を更新する処理は別に進みます。



地図の準備

分解・出題のジョブが並行して動きます。画面の歩行は処理状況の演出です。

単位	意味
セッション	テーマ、知識ツリー、回答根拠、自己評価、会話、処理状態を保持する探索の単位。
地図・系統	地図は探索結果の単位。系統は派生した地図や未探索候補をまとめる単位。
ノード / concept_id	知識項目を識別するID。理解度や問題の対象を結びつける。
エッジ	親子のpart関係など。複数の親を持つ結合も表現する。
ジョブ / revision	仕事の進捗と、更新の新旧を識別する情報。画面の見た目だけで完了を判断しない。

主要な構成

frontend/src/Application.tsxが探索操作をまとめ、api.tsがBearer認証付き通信とストリームを扱います。backend/app/main.pyがAPI入口、servicesが処理、policiesが評価・集計を担当します。保存先は設定によりFirestoreまたはSQLiteです。

> 基準：2026-10-13 / develop_v2 / 2026-10-13更新版。設定値はソース上の既定値であり、公開環境の環境変数を検査した値ではありません。APIキー等の秘密情報は本資料に含めません。

初期分解・先読み・展開の違い

動作・ロジック解説 / 02　宝箱が開くまで

01	02	03	04
セッション保存	初期地図を生成	次の分解を先読み	必要な枝を展開



queued

処理の順番待ち。生成そのものは始まっていません。



generating

AIで下位項目を準備している状態。



分解ready

分解結果は準備済み。問題readyとは独立です。



分解ready＋問題ready

両方の準備済みが開いた宝箱の条件です。

開始直後の受付を待たせない仕組み

entry_mode=map_chatではセッションを保存し、その結果を先に返します。レスポンス後のBackgroundTaskで初期生成を開始します。受付で話しながら、初期地図を作れる構成です。

先読みと公開は別

先読みは子の分解結果をキャッシュ等に準備します。展開はその結果を知識ツリーへ反映し、画面で子を表示する処理です。既に準備済み・実行中の生成は再利用し、同じ対象の生成をむやみに二重に始めない実装です。

既定の範囲と上限

初期の最大子数は5、追加分解の最大子数は4。自動先読みは先の1層まで、先読みの同時実行設定は10です。実際の数は計画、末端判定、設定、処理結果で変わります。固定の待ち時間を保証する値ではありません。

> 主な参照：main.py:start、services/diagnostic.py:schedule_initial_map・prepare_concept・schedule_prefetch、config.py、DecompositionIcon.tsx。

会話を地図に反映する流れ

動作・ロジック解説 / 03　受付・記録・評価の協調

01	02	03	04	05
発言を保存	受付が返答	根拠を抽出	配置・追加・分解	理解を評価



抽出と配置

発言の引用を付けた情報を取り出し、既存ノードや追加先との対応を調べます。



会話から評価

対応する評価観点について、発言から確認できた内容を評価します。

なぜ返答後も動くのか

会話の返答はストリームで返します。地図への反映は背景処理のキューで進むため、返答完了と地図更新完了の時刻は一致しません。会話の情報が複数の項目に対応すると、配置・分解・評価が複数回必要になります。

根拠を追跡する

発言を情報単位に整理し、メッセージIDと引用を使って対応付けます。未解決の配置先、追加予定、詳細化が必要な項目をキューに残し、地図が変わった際に再調整します。同じ発言をもう一度調べただけで根拠の確かさが増えないよう、評価で重複を扱います。

自動で表示する条件

画面はchat.created_concept_idsとopened_concept_idsを開く対象に加え、すべての祖先も開きます。後から子が到着しても表示へつながります。概念統合でIDが変わった場合はconcept_redirectsをたどります。

無制限に生成するわけではない

設定には1回の背景処理で行う分解・追加の上限があります。既定は各10件。話した単語すべてをノード化することや、すべての子孫を無限に展開することを保証する機能ではありません。

> 参照：services/chat.py、chat_queue.py、chat_map.py、chat_retrieval.py、frontend/src/atlas-session.ts。

理解度を算出するロジック

動作・ロジック解説 / 04　直接評価・暫定値・親の集計



自己評価

high=.90、medium=.55、low=.20。確かさ=.25。



会話由来の補完

使える評価がない対象にscore=.20、confidence=.10の暫定値を補います。

段階	処理
会話評価がある	評価観点ごとに会話と対応する問題根拠を組み合わせる。会話の自己申告は確かさを最大.25に制限。
会話評価がない・問題根拠がある	採点可能な回答のscoreを平均。採点不能は除く。
自己評価のみ	90 / 55 / 20%を使う。「問題で確認する」はそれ自体を得点にしない。
根拠なし	原則は未確認。チャット由来と識別できる非ルートとその自動展開枝は、条件を満たすと暫定値で補完。

会話と問題の重み

観点内では新しい根拠から順に0.6の累乗で減衰させ、各根拠の確かさも重みに使います。主要観点は重み1、補助観点は0.5。直近の得点が大きく矛盾すると確かさを下げます。これは単純な「最新値で上書き」とは異なります。

親の理解度

直下の子がすべて評価済みなら、子側の寄与元を集めて平均します。共有ノードを二重に数えません。未評価の子が残る場合は親自身の値を使い、親にも値がなければ未確認のままになる場合があります。

> 例：子の寄与元が20%と80%なら親は50%。子が増えると集計条件や平均が変わり、親の表示も変わり得ます。参照：policies/conversation.py:estimates、understanding.py:summaries。

問題・自己評価・ノード別の進捗

動作・ロジック解説 / 05　操作から表示更新まで

01	02	03	04
対象IDを確定	保存・生成を依頼	サーバー結果	対象ノードへ反映

自己評価の処理順

Application.tsxのassessSelectedは、session_idとconcept_idの組を進捗のキーにします。self-assessmentを送信し、成功後に、unsureならselectで問題へ、その他ならexpandConceptで下位展開へ進みます。保存失敗時に次の処理へ進めません。

問題ページの対象

next_question、pending_review、reassessment等のconcept_idが選択ノードと一致するかを判定します。問題確認要求、フィードバック等の対象状態からタブ表示を決めます。APIのアクティブな問題は1つであり、各ノードで独立した問題を同時回答するUIではありません。

準備済みなのに待つ理由

出題の準備状態と、その問題を現在の回答対象にする処理は別です。認証付き通信、保存、状態取得や画面反映に時間がかかることがあります。生成前なら待ち行列とAI応答も加わります。宝箱はこれら全工程の終了通知ではありません。



問題だけ準備済み

分解が未準備なら小さな答案で表現します。



失敗は対象ごと

分解失敗と問題生成失敗は別状態。右下の小印は問題側の進捗です。

画面が更新を取り込む方法

操作の応答やSSEに加え、探索状態を定期取得します。生成中は800ms、通常は2000ms。ChatPanelの背景更新は1200msです。ネットワークやサーバー時間が加わるため、この間隔が操作全体の所要時間ではありません。

> 古いrevisionの応答で新しい地図を巻き戻さないようにreconcileSessionで比較します。参照：Application.tsx、api.ts、ChatPanel.tsx、atlas-session.ts。

配置・等高線・調査隊の動き

動作・ロジック解説 / 06　地図表現の規則



開始地点

基本となる中心。新しい子は発生元から中心の外側へ配置します。



待機人数

役割の最大数から稼働数を引く。順番待ちは開始されるまで出動扱いになりません。



情報の引き継ぎ

新たに届いた引き継ぎイベントの演出。過去の全履歴を繰り返すものではありません。



受付係の居場所

探索受付のアイコンで動作します。通常の地図上作業係とは配置が異なります。

力方向グラフ

D3のリンク力、反発、衝突回避、中心への力、深さに応じた半径方向の力を組み合わせます。既存座標を引き継ぎ、新規の子は親から外側へ初期配置します。ドラッグ中は位置を固定し、離すと固定を解除します。

等高線

開始点からの最短ホップ数をBFSで計算します。この距離はエッジを両方向にたどった接続距離です。画面上では距離の重み付き補間と外側への補正で場を作り、等値線を描きます。数学的に正確な地理の標高線ではありません。

人の数と仕事の数

ジョブの役割・段階・対象を基に配置します。並列の仕事が同じノードに集まる場合があります。記録係の同じ対象・同じ段階の処理はまとめて表示する場合があり、見える人物数をAPI呼び出し回数と同一視できません。

> 参照：map-geometry.ts、LiveAtlasMap.tsx、MapPages.tsx、live-map-jobs.ts、map-crew.ts、MapAgents.tsx、AgentHandoffs.tsx。

保存・認証・障害の調べ方

動作・ロジック解説 / 07　運用と実装を読む入口

調べたいもの	主なコード
API・所有者の確認	backend/app/main.py、auth.py、services/maps.py、repositories/*
初期生成・分解・問題・採点	services/diagnostic.py、agents/*、prompts/*
会話・項目追加・再処理	services/chat.py、chat_queue.py、chat_map.py
理解度・統合・削除	policies/understanding.py、conversation.py、services/concept_merge.py・concept_deletion.py
画面と状態整合性	Application.tsx、atlas-session.ts、api.ts、MapPages.tsx
モデル・並列数・保存先	backend/app/config.py、llm/adk.py、llm/vertex_auth.py

認証と保存

フロントはBearerトークンを送信します。APIは認証ユーザーに紐づく一覧や所有権を確認します。401を受けるとフロント側の認証状態をクリアします。JSONダウンロードはサーバー保存とは別の書き出しです。

障害を切り分ける順序

①対象の地図とノードID、操作、時刻を特定 → ②分解と問題のどちらが待機・失敗か確認 → ③APIの状態コードと完了結果を確認 → ④背景ジョブと保存状態を確認 → ⑤AI呼び出しや認証・設定を確認。APIキー・トークン・個人の会話本文を不用意に共有しないでください。

既定モデルについて

ソースの既定値はgemini-3.5-flash-liteです。環境変数で上書き可能なので、この記載だけで公開サーバーが同モデルを使っているとは断定できません。利用経路・認証もデプロイ設定に依存します。

> この資料はコードを読んで実装を説明したものです。エラーが起きないことや、生成内容の正しさを保証するものではありません。今回の作成ではアプリの動作を変更せず、マニュアルのレイアウトと出力を検証しています。